Magenable CouponExpiry — Merchant and Administrator Guide

1. Purpose

Magenable CouponExpiry adds rolling validity to individually generated cart-price-rule coupons.

Instead of every coupon sharing one calendar expiry date, each managed coupon expires a configured number of consecutive 24-hour days after it is generated.

Example:

- Rule validity: 14 days
- Coupon generated: 1 September at 10:00 UTC
- Coupon expires: 15 September at 10:00 UTC

A coupon remains subject to all normal Magento restrictions. Rolling validity does not override an inactive rule, rule start/end dates, website assignments, customer groups, usage limits, cart conditions, priorities or discount actions.

2. Supported coupon type

Rolling expiry is available for cart price rules configured as:

- **Coupon:** Specific Coupon
- **Use Auto Generation:** enabled

Manually entered shared primary coupon codes are not managed by this version.

3. Configure a cart price rule

1. Sign in to the Magento Admin.
2. Open **Marketing → Promotions → Cart Price Rules**.
3. Create a rule or open an existing rule.
4. Under **Rule Information**, configure the normal Magento fields:
5. Rule Name
6. Active
7. Websites
8. Customer Groups
9. Native rule dates and usage limits
10. Set **Coupon** to **Specific Coupon**.
11. Enable **Use Auto Generation**.

12. Enter **Coupon validity (days)**.
13. Complete the Conditions and Actions sections normally.
14. Save the rule before generating coupons.

Coupon validity field

Allowed values:

- Blank: future generated coupons have no additional rolling expiry.
- `1` through `3650`: each future generated coupon receives that many consecutive 24-hour periods.

Rejected values include zero, negative values, fractions, text and values above 3650.

A populated validity setting cannot be saved on an incompatible coupon mode. Clear the field first if auto generation is being disabled.

4. Generate coupons in Admin

1. Save the eligible cart price rule.
2. Open its **Manage Coupon Codes** section.
3. Enter the desired native generation settings, including quantity, length, format, prefix, suffix and delimiter.
4. Select **Generate**.

Magento places admin generation requests into the native `sales_rule.codegenerator` queue. The request is complete only after `codegeneratorProcessor` consumes it.

The site's operations team must run Magento cron and message-queue consumers through the deployment's normal process manager. Newly requested coupons normally appear after `codegeneratorProcessor` has completed the message. Refresh the Manage Coupon Codes grid after the documented queue interval.

Do not submit the same generation request repeatedly merely because coupons are not immediately visible. Native queued generation is not idempotent; processing two submitted requests creates two batches.

5. Understand issued coupon data

The Manage Coupon Codes grid retains Magento's native columns and adds:

- **Issued At** — time the coupon was created
- **Rolling Expiry** — coupon-specific expiry
- **Coupon Validity Status** — time-only status

Issued and expiry times are stored in UTC and displayed in the configured Admin timezone. The timezone is included in the column labels.

Status values

Status	Meaning
Within coupon validity	Current time is before the coupon's rolling expiry. Other Magento restrictions may still prevent use.
Expired	Current time is equal to or later than rolling expiry.
Not managed	The coupon has no Magenable CouponExpiry metadata, usually because it predates the module or was created under a rule without rolling validity.

“Within coupon validity” does not mean the coupon is guaranteed to work. The parent rule must still pass native Magento validation.

The additional columns and values are included in CSV and Excel XML exports.

6. Policy changes

Changing **Coupon validity (days)** affects future coupons only.

- Existing coupons retain their original issue time, validity snapshot and expiry.
- Reducing the rule value does not shorten existing coupons.
- Increasing the rule value does not extend existing coupons.
- Clearing the field does not remove expiry from already managed coupons.
- Existing coupons are not backfilled when rolling validity is enabled.

Issued coupon metadata is intentionally read-only. Normal coupon updates cannot move a managed coupon to another cart price rule or alter its rolling expiry.

7. Customer experience at expiry

Coupon expires before being added

Magento rejects it as an invalid coupon. No discount is applied.

Coupon expires while already in the cart

On the next totals/cart validation:

1. The coupon discount is removed.
2. Totals are recalculated.
3. The customer is shown an expiry notice and asked to review the updated totals.

Coupon expires during checkout

If discounted totals were accepted before expiry, Place Order is blocked. The customer must return to the cart, review the notice and updated totals, and submit checkout again.

The module does not silently place an order at a higher total without that review step.

Historical completed orders, invoices, credit memos and refunds are not modified.

8. Reorders

Magento's native reorder process did not automatically copy the historical coupon in the verified target configuration.

If a customer attempts to apply the old managed coupon again, its current expiry and all current parent-rule restrictions are evaluated. An expired coupon cannot recreate the historical discount.

The original order remains unchanged.

9. Website and store scope

A rule can issue coupons only for its assigned websites. Administrators should verify the **Websites** selection on the cart price rule before generation.

A rule intended for multiple storefronts must include every relevant website. Coupon expiry is enforced by server-side SalesRule and quote submission services, not only by frontend JavaScript, so supported themes use the same policy.

10. External system issuance

The module also provides authenticated integration APIs for idempotent issuance and managed-coupon lookup. These are intended for system integrations, not normal merchant browser use.

Integration administrators should grant only the required ACL resource:

- CouponExpiry Read
- CouponExpiry Issue

See `API.md` for endpoint, authentication, replay, conflict, pagination and error behavior.

Never put customer email addresses or other personal data into API request IDs.

11. Troubleshooting

Coupons remain “in queue” or do not appear

1. Wait for at least one cron interval and refresh the grid.
2. Confirm that only one generation request was submitted.
3. Ask an operator to verify that Magento cron and `codegeneratorProcessor` are running.
4. Do not resubmit until the pending queue message has been checked.

An operator can check the registered consumer and, when appropriate for the site's process-management policy, execute a bounded diagnostic run:

```
bin/magento queue:consumers:list
bin/magento queue:consumers:start codegeneratorProcessor --single-thread --
max-messages=100
```

Do not start an unmanaged duplicate consumer when the deployment already runs one through Supervisor, systemd, Kubernetes, Commerce Cloud or Magento's cron consumer runner.

Coupon says “Within coupon validity” but does not apply

Check the parent cart price rule:

- Active status
- Start/end dates
- Website assignment
- Customer groups
- Conditions and Actions
- Uses per coupon/customer
- Priority and Stop Further Rules Processing

The status column reports rolling time validity only.

Customer sees an expiry/review message

This is expected when a previously accepted discount expires. The customer should review the cart totals and continue checkout again. Do not bypass the notice by editing quote/order database records.

Managed metadata appears inconsistent

Stop issuing affected coupons and escalate to a technical administrator. Do not edit issue or expiry timestamps manually. Corrupt managed metadata fails closed intentionally.

12. Safe administration rules

- Do not edit managed coupon expiry directly in the database.
- Do not move a managed coupon to another rule.
- Do not delete idempotency records to force another API issue.
- Do not disable the module while managed parent rules remain active; doing so can revive expired native coupon codes.
- Deactivate affected rules before any emergency rollback.
- Retain expired coupons and metadata for reporting unless a documented retention policy is approved.

Technical deployment, monitoring, integrity checks and rollback steps are documented in `OPERATIONS.md`.

13. Current compatibility statement

Version 1.0.4 has the same runtime code as 1.0.3 and has been tested and supported on:

- Magento Open Source 2.4.7-p10
- Magento Open Source 2.4.8-p5
- Magento Open Source 2.4.9
- Adobe Commerce 2.4.8, confirmed in a seller-provided test environment
- Mage-OS 3.5.0, based on Magento 2.4.9
- PHP 8.3, 8.4 and 8.5 on the corresponding verified open-source targets
- Luma storefront and open-source Hyvä Theme with the standard Magento/Luma checkout fallback
- Guest and registered checkout, REST and GraphQL submission, native Admin order creation and native customer reorder
- Database-backed and RabbitMQ-backed native Magento queues

The exact Adobe Commerce 2.4.8 patch level, PHP version and infrastructure topology were not supplied for the public test report. Adobe Commerce Cloud, other Adobe Commerce versions, custom checkouts, multi-coupon implementations, alternative queue topologies and optional Commerce-only features require separate qualification. See `docs/COMPATIBILITY.md` for the product support matrix. Qualification on a merchant's actual checkout, payment and queue configuration is required before production deployment.

Installation and Upgrade

Requirements

- A compatible Magento Open Source, Adobe Commerce or Mage-OS release listed in `COMPATIBILITY.md`.
- PHP version supported by that platform release.
- Composer 2.
- A functioning Magento cron and message-queue consumer configuration.
- Database and application-file backups tested before deployment.

Composer installation

Configure the Composer repository supplied by Magenable or the marketplace, then run:

```
composer require magenable/module-coupon-expiry:^1.0
bin/magento module:enable Magenable_CouponExpiry
bin/magento setup:upgrade
```

For production mode, use the deployment sequence appropriate to the site, including dependency injection compilation and static-content deployment:

```
bin/magento setup:di:compile
bin/magento setup:static-content:deploy --force
bin/magento cache:flush
```

Do not run production deployment commands as an account that leaves Magento writable directories owned by the wrong user.

Verify installation

```
bin/magento module:status Magenable_CouponExpiry
bin/magento queue:consumers:list
bin/magento indexer:status
```

Expected results:

- `Magenable_CouponExpiry` is enabled.
- `codegeneratorProcessor` is present in the consumer list.
- Indexers are ready or completing normally.
- Admin and storefront pages render without exceptions.

Create a disposable eligible cart price rule, generate one coupon, allow the standard consumer to process it, and verify that the Manage Coupon Codes grid displays its issue and rolling-expiry values.

Upgrades

1. Review `CHANGELOG.md` and `docs/COMPATIBILITY.md`.
2. Back up the database and current package lock file.
3. Stop new coupon-issuance traffic for the deployment window.
4. Update the package:

```
composer require magenable/module-coupon-expiry:^1.0 --with-all-
dependencies
bin/magento setup:upgrade
bin/magento setup:di:compile
bin/magento setup:static-content:deploy --force
bin/magento cache:flush
```

1. Restore normal cron/consumer supervision.
2. Run the verification steps and a checkout smoke test.

Uninstall and rollback

Disabling the PHP module can allow otherwise-valid native coupon codes to bypass rolling-expiry enforcement. Do not disable or remove it while managed parent rules remain active.

Use the non-destructive rollback procedure in `OPERATIONS.md`. This release intentionally does not drop audit metadata or idempotency tombstones automatically.

License

Magenable CouponExpiry is released under the MIT License. See `LICENSE.md` in the package for the complete license text.